

TAKE-HOME PROJECT MANUAL

Agentic Incident Attribution and Safe Remediation Platform

Engineering internship project using LangGraph, MCP, Temporal, PostgreSQL and Kubernetes.

LANGGRAPH	AGENTIC AI	MCP SERVERS	TEMPORAL.IO
POSTGRESQL	TYPESCRIPT / NODE.JS	KUBERNETES	GOLANG

Submission standard

The solution must be hosted on a cloud platform, remain accessible for evaluation, and be submitted through a GitHub repository containing the source code, deployment configuration, documentation, tests and CI workflow.

PROJECT MANUAL

Contents

01 Project Overview	10 Local Development Support
02 Company and Application Context	11 Testing Requirements
03 Project Objective	12 Documentation Requirements
04 Incident Scenario	13 Required Submission
05 Target System Behaviour	14 Evaluation Walkthrough
06 Implementation Manual	15 Evaluation Criteria
07 Technology Choices	16 What Will Be Evaluated
08 Kubernetes and Cloud Deployment	17 Bonus Enhancements
09 Hosted Solution Requirements	18 Final Submission Checklist

PROJECT AT A GLANCE

BUILD	An agentic incident-attribution and safe-remediation platform.
USE	LangGraph, MCP, Temporal, PostgreSQL and Kubernetes.
HOST	Deploy the solution on a cloud platform of the candidate's choice.
SUBMIT	Provide the hosted URL, GitHub repository, documentation, tests and CI.
PROTECT	Require human approval before any remediation is executed.

Core safety requirement

The platform may recommend a remediation, but no remediation may execute until a human approver explicitly approves it through a Temporal signal.

1. Project Overview

Build and host a working prototype that uses Agentic AI, LangGraph, MCP, Temporal, PostgreSQL, TypeScript/JavaScript or Golang, and Kubernetes to investigate incidents in a cloud-native application.

The system must identify the infrastructure or application change most likely to have caused an incident, explain the evidence supporting that conclusion, recommend a safe remediation, and require human approval before executing any action.

This is a hands-on engineering project. The submitted solution must:

1. Be deployed and accessible through a hosted environment.
2. Be checked into GitHub.
3. Include setup, deployment, testing and evaluation instructions.
4. Demonstrate the complete incident-analysis and approval workflow.

Completion condition

A solution that runs only on the candidate's local machine will not be considered complete.

2. Company and Application Context

Acme Inc. operates a mid-scale e-commerce platform serving more than two million monthly active users.

The platform runs on AWS using a cloud-native Kubernetes architecture:

- React and TypeScript storefront
- Python microservices
- PostgreSQL
- Redis
- Amazon SQS and SNS
- Amazon S3
- HashiCorp Vault
- GitHub Actions
- Amazon EKS

Use the supplied Acme application architecture diagram as the reference architecture for this project.

COMPANY CONTEXT

Acme Inc operates a mid-scale e-commerce platform serving 2M+ monthly active users. The platform runs on **AWS** with a cloud-native Kubernetes architecture: a **React/TypeScript** storefront, **Python** microservices backend, **PostgreSQL** and **Redis** data stores, **SQS/SNS** queues for async processing, **S3** for media assets, **HashiCorp Vault** for secrets, and **GitHub Actions** CI/CD pipelines deploying to **EKS**.

APPLICATION ARCHITECTURE (FROM THE OVERLAY)

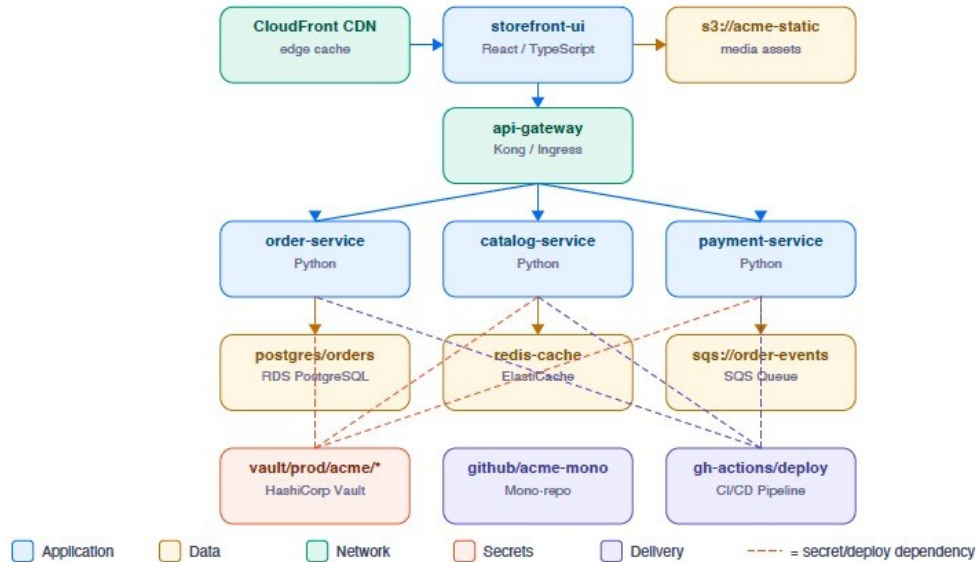


Figure 1. Reference architecture for the Acme e-commerce platform.

The main application components are:

```

CloudFront CDN
|
storefront-ui
|
api-gateway
|
+-- order-service
|   |-- postgres/orders
|   |-- redis-cache
|   |-- vault/prod/acme/*
|   |-- gh-actions/deploy
|
+-- catalog-service
|   |-- redis-cache
|   |-- vault/prod/acme/*
|   |-- gh-actions/deploy
|
+-- payment-service
|   |-- sqs/order-events
|   |-- vault/prod/acme/*
|   |-- gh-actions/deploy

```

3. Project Objective

Build an Incident Attribution Agent that:

1. Receives an application incident.
2. Retrieves recent application and infrastructure changes.
3. Uses the architecture graph to identify related resources.
4. Ranks changes that may have caused the incident.
5. Uses a LangGraph agent to produce an evidence-based conclusion.
6. Recommends a safe next action.
7. Starts or continues a Temporal workflow.

8. Waits for human approval.
9. Executes a simulated remediation.
10. Records all system, agent, tool, workflow and human actions in an audit trail.

The candidate must host the completed solution on a cloud platform of their choice.

- AWS
- Microsoft Azure
- Google Cloud
- DigitalOcean
- Oracle Cloud
- Render
- Railway
- Fly.io
- Another suitable cloud or Kubernetes hosting platform

Cloud-provider neutrality

The choice of cloud provider will not affect the evaluation. The quality of the implementation, deployment, documentation and engineering decisions will be evaluated.

4. Incident Scenario

Acme's monitoring system detects an increase in order-processing failures.

Sample Incident

```
{
  "incident_id": "INC-1042",
  "service": "order-service",
  "environment": "production",
  "symptom": "Order creation failure rate increased from 1% to 18%",
  "detected_at": "2026-08-04T08:15:00Z"
}
```

Recent Changes

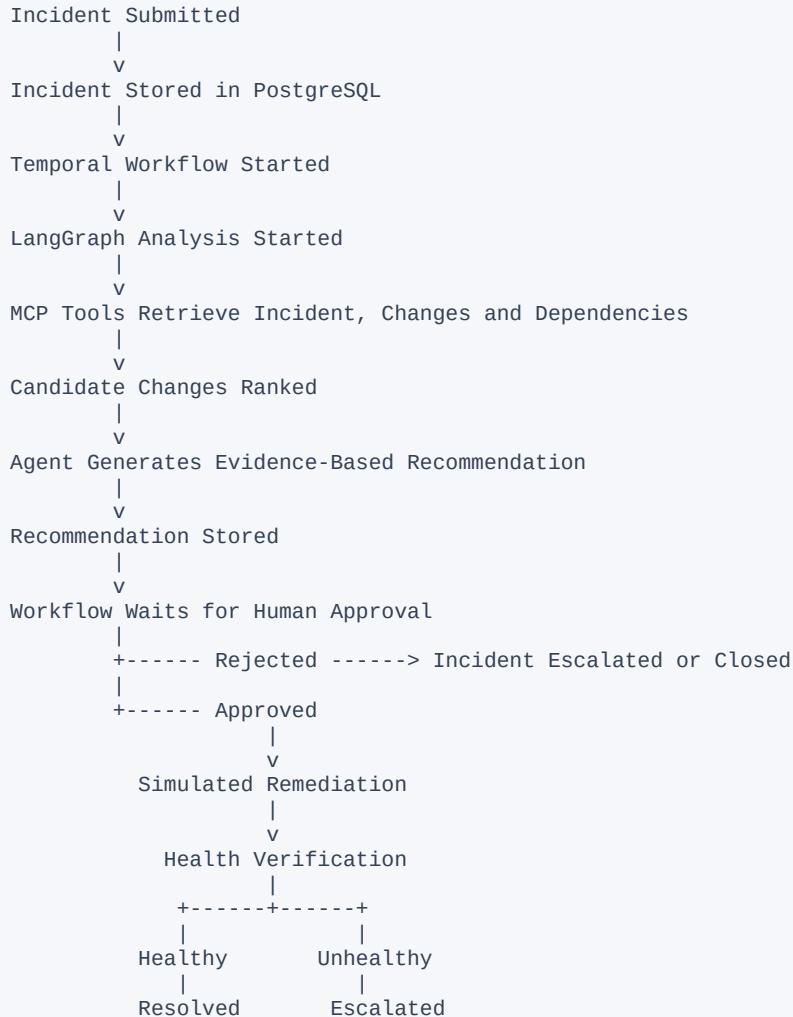
```
[
  {
    "change_id": "CHG-201",
    "resource": "order-service",
    "type": "deployment",
    "version": "v2.4.1",
    "deployed_at": "2026-08-04T08:05:00Z",
    "pipeline": "gh-actions/deploy"
  },
  {
    "change_id": "CHG-202",
    "resource": "redis-cache",
    "type": "configuration",
    "description": "Reduced connection timeout",
    "deployed_at": "2026-08-04T07:55:00Z"
  },
  {
    "change_id": "CHG-203",
    "resource": "catalog-service",
    "type": "deployment",
    "version": "v1.9.0",
    "deployed_at": "2026-08-04T06:30:00Z"
  }
]
```

The system should determine which change is most likely to have caused the incident and recommend an appropriate next action.

The correct result does not need to be hard-coded. The solution should demonstrate how the conclusion was reached using the incident, recent changes, dependency relationships, time correlation and other available evidence.

5. Target System Behaviour

The completed system should support the following end-to-end flow:



6. Implementation Manual

Complete the following implementation steps. Candidates may organise the work differently, provided all mandatory outcomes are present and demonstrable.

Step 1: Create the GitHub Repository

Create a GitHub repository for the project. The repository may be public or private with access granted to the evaluators.

Use a clear repository name, such as:

```
agentic-incident-attribution
```

The suggested repository structure is:

```
README.md
LICENSE
```

```
.env.example

architecture/
  architecture-diagram.*
  sequence-diagram.*
  decisions/

services/
  api/
  agent/
  mcp-server/
  temporal-worker/

database/
  migrations/
  seeds/

deploy/
  docker-compose.yml
  kubernetes/
  helm/

tests/
  integration/
  end-to-end/

sample-data/
  incidents.json
  changes.json
  dependencies.json

docs/
  api.md
  deployment.md
  demo.md
```

Candidates may use a monorepo or separate packages within the same repository.

Repository security

The repository must not contain live credentials, API keys, database passwords or cloud secrets.

Step 2: Model the Architecture Dependency Graph

Represent the supplied architecture as a graph. At minimum, the graph should support:

- Applications
- Services
- Databases
- Caches
- Queues
- Secrets
- Source-code repositories
- Deployment pipelines
- Direct dependencies
- Indirect dependencies

Example representation:

```
{
  "order-service": [
    "api-gateway",
    "postgres/orders",
    "redis-cache",
    "vault/prod/acme/*",
    "github/acme-mono",
    "gh-actions/deploy"
  ],
  "catalog-service": [
    "api-gateway",
    "redis-cache",
    "vault/prod/acme/*",
```

```

    "github/acme-mono",
    "gh-actions/deploy"
  ],
  "payment-service": [
    "api-gateway",
    "sqs/order-events",
    "vault/prod/acme/*",
    "github/acme-mono",
    "gh-actions/deploy"
  ]
}

```

The graph may be implemented using PostgreSQL tables, JSON configuration, an in-memory graph, a graph database, or another clearly documented approach.

The system must be able to answer questions such as:

```

What resources does order-service depend on?
Which services are affected by redis-cache?
Is catalog-service directly or indirectly related to order-service?
Which deployment pipeline can modify order-service?

```

Step 3: Create the PostgreSQL Data Model

Use PostgreSQL as the system of record. Database migrations must be included in the repository and must be executable through a documented command.

Incidents

```

id
service
environment
symptom
detected_at
status
created_at
updated_at

```

Suggested status values:

```

RECEIVED
ANALYSING
AWAITING_APPROVAL
APPROVED
REJECTED
REMEDIATING
RESOLVED
ESCALATED
FAILED

```

Changes

```

id
resource
change_type
description
version
deployed_at
source
metadata
created_at

```

Resources

```

id
name
resource_type
environment

```



```
metadata
created_at
```

Resource Relationships

```
id
source_resource_id
target_resource_id
relationship_type
created_at
```

Example relationship types:

```
DEPENDS_ON
DEPLOYED_BY
READS_FROM
WRITES_TO
USES_SECRET
PUBLISHES_TO
CONSUMES_FROM
HOSTED_ON
```

Analysis Results

```
id
incident_id
probable_change_id
confidence_score
reasoning_summary
evidence
recommended_action
analysis_status
created_at
```

Approval Requests

```
id
incident_id
analysis_id
requested_action
status
requested_at
responded_at
responded_by
comment
```

Audit Events

```
id
incident_id
workflow_id
actor_type
actor_name
action
input
output
status
timestamp
```

Example actor types:

```
USER
AGENT
MCP_TOOL
TEMPORAL_WORKFLOW
TEMPORAL_ACTIVITY
SYSTEM
```

JSON or JSONB may be used for structured metadata, evidence, tool input and tool output.

Step 4: Build the Backend API

The backend API may be implemented using Golang or TypeScript with Node.js.

Health Endpoints

```
GET /health
GET /ready
```

Incident Endpoints

```
POST /api/incidents
GET /api/incidents/:id
GET /api/incidents
```

Analysis Endpoints

```
POST /api/incidents/:id/analyse
GET /api/incidents/:id/analysis
GET /api/incidents/:id/candidates
```

Approval Endpoints

```
POST /api/incidents/:id/approve
POST /api/incidents/:id/reject
```

Workflow Endpoint

```
GET /api/incidents/:id/workflow
```

Audit Endpoint

```
GET /api/incidents/:id/audit-log
```

The API must include:

- Input validation
- Structured error responses
- Request IDs or correlation IDs
- Appropriate HTTP status codes
- Structured application logging
- Database connection handling
- Graceful shutdown
- Configuration through environment variables
- Protection against duplicate analysis requests
- Idempotency for approval and remediation requests

A suggested error format is:

```
{
  "error": {
    "code": "INCIDENT_NOT_FOUND",
    "message": "Incident INC-1042 was not found",
    "request_id": "req-12345"
  }
}
```

Step 5: Implement Candidate-Change Ranking

Before calling the LLM, implement deterministic candidate filtering and ranking.

The ranking should consider factors such as:

- Time between the change and incident
- Whether the changed resource is the affected service
- Dependency distance from the affected service
- Change type
- Environment
- Deployment or configuration source
- Whether multiple services depend on the changed resource
- Availability of supporting or conflicting evidence

Example scoring factors:

Direct change to affected service	+40
Change to a direct dependency	+25
Change to an indirect dependency	+10
Change occurred within 15 minutes	+25
Change occurred within 60 minutes	+15
Change occurred in the same environment	+10
Unrelated resource	-30
Change occurred several hours earlier	-15

The exact scoring model is up to the candidate. The candidate must document:

- The factors used
- The score assigned to each factor
- How scores are normalised
- How the final confidence value is calculated
- Which parts are deterministic
- Which parts are handled by the LLM

Engineering expectation

The LLM should not be solely responsible for discovering or ranking every candidate.

Step 6: Build the MCP Server

Build an MCP server that exposes tools to the LangGraph agent. The agent must use MCP tools to obtain operational data. It must not bypass the MCP layer and directly query the database for its analysis.

Required MCP Tools

```
get_incident
get_recent_changes
get_resource_dependencies
record_analysis_result
```

Recommended MCP Tools

```
get_ranked_candidate_changes
get_change_details
get_deployment_details
get_service_health
get_secret_metadata
create_approval_request
get_approval_status
start_simulated_remediation
record_audit_event
```

A tool response should be structured and machine-readable. Example:

```
{
  "incident_id": "INC-1042",
```

```

    "service": "order-service",
    "related_resources": [
      {
        "resource": "redis-cache",
        "relationship": "DEPENDS_ON",
        "distance": 1
      },
      {
        "resource": "gh-actions/deploy",
        "relationship": "DEPLOYED_BY",
        "distance": 1
      }
    ]
  }
}

```

Each MCP tool must:

- Validate its input
- Return structured errors
- Record meaningful audit information
- Avoid exposing credentials or secret values
- Be independently testable

Secret-handling rule

For secret resources, return only metadata. Do not return the contents of secrets.

Step 7: Build the LangGraph Agent

Implement the incident-analysis workflow using LangGraph. A suggested graph is:

```

Load Incident
  |
  v
Find Related Resources
  |
  v
Retrieve Recent Changes
  |
  v
Rank Candidate Changes
  |
  v
Evaluate Evidence
  |
  v
Generate Explanation
  |
  v
Recommend Action
  |
  v
Validate Recommendation
  |
  v
Store Analysis Result
  |
  v
Request Human Approval

```

The graph must demonstrate:

- Explicit state management
- MCP tool invocation
- Conditional branching
- Error handling
- Retry or recovery paths
- A maximum iteration limit
- Human-in-the-loop behaviour
- Separation of analysis and action execution

The graph should have branches for situations such as:

- No relevant changes found
- Low-confidence conclusion
- MCP tool failure
- Invalid LLM output
- Unsupported remediation
- Human approval required
- Manual investigation required

The LangGraph state may contain fields such as:

```
interface IncidentAnalysisState {
  incidentId: string;
  incident?: Incident;
  dependencies: ResourceDependency[];
  recentChanges: Change[];
  rankedCandidates: RankedCandidate[];
  evidence: EvidenceItem[];
  probableChangeId?: string;
  confidence?: number;
  reasoningSummary?: string;
  recommendation?: RemediationRecommendation;
  errors: WorkflowError[];
  iterationCount: number;
}
```

The final response should resemble:

```
{
  "incident_id": "INC-1042",
  "status": "AWAITING_APPROVAL",
  "probable_cause": {
    "change_id": "CHG-201",
    "resource": "order-service",
    "confidence": 0.87
  },
  "evidence": [
    {
      "type": "TEMPORAL_CORRELATION",
      "description": "The deployment occurred 10 minutes before the incident."
    },
    {
      "type": "DIRECT_RESOURCE_MATCH",
      "description": "The changed resource is the service experiencing the failure."
    },
    {
      "type": "DEPLOYMENT_CORRELATION",
      "description": "Version v2.4.1 was the most recent order-service deployment."
    }
  ],
  "reasoning_summary": "The order-service deployment is the strongest candidate because it directly modified the affected service shortly before the failure rate increased.",
  "recommended_action": {
    "type": "ROLLBACK",
    "resource": "order-service",
    "from_version": "v2.4.1",
    "to_version": "v2.4.0",
    "execution": "SIMULATED",
    "requires_approval": true
  }
}
```

Reasoning-output rule

Do not expose or store raw hidden chain-of-thought. Store only structured evidence, candidate scores, concise reasoning summaries, recommendations and audit-appropriate tool inputs and outputs.

Step 8: Implement the Temporal Workflow

Use Temporal to orchestrate the durable incident lifecycle. The workflow should resemble:

```
Incident Received
  |
  v
```



Temporal Requirements

- Survive worker restarts
- Use a Temporal signal for approval
- Use a Temporal signal for rejection
- Expose a query for workflow status
- Use activity timeouts
- Use appropriate retry policies
- Keep workflow code deterministic
- Make activities idempotent
- Record workflow state changes in the audit log
- Move failed or unsuccessful remediations to ESCALATED

Suggested Temporal Signals

```

approveRemediation
rejectRemediation
cancelIncident
  
```

Suggested Temporal Queries

```

getStatus
getRecommendation
getApprovalStatus
getTimeline
  
```

Simulated Remediation

No real production change is required. Example:

```

{
  "action": "rollback",
  "service": "order-service",
  "from_version": "v2.4.1",
  "to_version": "v2.4.0",
  "execution": "simulated",
  "result": "success"
}
  
```

The simulated remediation should still behave like a real operation:

- Validate the target service

- Verify that approval exists
- Prevent duplicate execution
- Record the requested action
- Return a structured result
- Support a controlled failure scenario
- Trigger a simulated health check

Step 9: Add Human Approval

Mandatory approval gate

A production remediation must never execute automatically.

The API should allow an evaluator to approve the recommendation:

```
POST /api/incidents/INC-1042/approve
Content-Type: application/json

{
  "approved_by": "evaluator@example.com",
  "comment": "Rollback approved after reviewing the evidence."
}
```

The API must send the approval to the running Temporal workflow as a signal.

The solution must also support rejection:

```
POST /api/incidents/INC-1042/reject
Content-Type: application/json

{
  "rejected_by": "evaluator@example.com",
  "comment": "Confidence is too low. Escalate for manual investigation."
}
```

Repeated approval or rejection requests must be handled safely and idempotently.

Step 10: Implement the Audit Trail

Every important action must be recorded. The audit trail should include:

- Incident creation
- Analysis request
- Temporal workflow start
- LangGraph node transitions
- MCP tool calls
- Candidate ranking
- LLM request metadata
- LLM response summary
- Recommendation generation
- Approval request
- Human approval or rejection
- Remediation attempt
- Health verification
- Workflow completion or escalation

Example response:

```
{
  "incident_id": "INC-1042",
  "events": [
    {
      "actor_type": "USER",
      "actor_name": "api-user",
      "action": "INCIDENT_CREATED",
      "timestamp": "2026-08-04T08:16:00Z"
    },
    {
      "actor_type": "MCP_TOOL",
```

```
    "actor_name": "get_recent_changes",
    "action": "TOOL_EXECUTED",
    "timestamp": "2026-08-04T08:16:05Z"
  },
  {
    "actor_type": "AGENT",
    "actor_name": "incident-attribution-agent",
    "action": "ANALYSIS_COMPLETED",
    "timestamp": "2026-08-04T08:16:10Z"
  },
  {
    "actor_type": "USER",
    "actor_name": "evaluator@example.com",
    "action": "REMEDIATION_APPROVED",
    "timestamp": "2026-08-04T08:18:00Z"
  }
]
}
```

Audit-data hygiene

Sensitive values must be redacted before they are written to the audit log.

7. Technology Choices

Candidates may choose one of the following implementation approaches.

Option A: TypeScript-First

Component	Technology
API	Node.js and TypeScript
Agent	LangGraph.js
MCP Server	MCP TypeScript SDK
Temporal	Temporal TypeScript SDK
Database	PostgreSQL
Deployment	Kubernetes

Option B: Golang Backend

Component	Technology
API	Golang
Agent	LangGraph.js or another supported LangGraph implementation
MCP Server	TypeScript or Golang
Temporal	Temporal Go SDK or TypeScript SDK
Database	PostgreSQL
Deployment	Kubernetes

General Requirements

- TypeScript is preferred over plain JavaScript.
- The candidate may use an LLM provider of their choice.

- Automated tests must not require a live paid LLM.
- LLM responses should be mocked or replaced with a deterministic test implementation during CI.
- All technology choices and trade-offs must be documented.

8. Kubernetes and Cloud Deployment

The completed application must be hosted on a cloud platform selected by the candidate.

Kubernetes manifests or a Helm chart must be included for:

- API service
- LangGraph agent service or worker
- MCP server
- Temporal worker
- PostgreSQL, or connectivity to managed PostgreSQL
- Temporal development server, self-hosted Temporal, or Temporal Cloud connectivity

The deployment must include, where applicable:

- Deployments
- Services
- Ingress
- ConfigMaps
- Secrets
- Liveness probes
- Readiness probes
- Resource requests
- Resource limits
- Persistent storage
- Database migrations
- Environment-specific configuration

Candidates may use:

- Managed Kubernetes
- A Kubernetes cluster deployed on a virtual machine
- k3s
- Another hosted Kubernetes-compatible environment

Managed PostgreSQL and Temporal Cloud may be used. Self-hosted alternatives are also acceptable.

Reproducibility requirement

The repository must contain sufficient deployment configuration to reproduce the hosted environment.

9. Hosted Solution Requirements

The hosted solution must remain available during the evaluation period.

The submission must provide:

Application/API URL
Health-check URL
GitHub repository URL
API documentation URL, if separate
Evaluator credentials, if authentication is enabled
Cloud and deployment architecture summary

At minimum, the evaluator must be able to:

1. Submit the sample incident.
2. Start the analysis.
3. Retrieve ranked candidate changes.
4. Retrieve the analysis result.
5. View the recommended action.

6. Approve or reject the action.
7. Retrieve the Temporal workflow status.
8. View the complete audit trail.

A graphical user interface is optional. A well-documented hosted API with Swagger, OpenAPI, Postman, Bruno or equivalent documentation is acceptable.

Credential submission

Do not commit evaluator passwords or cloud credentials to GitHub. Provide credentials through an appropriate secure submission channel.

10. Local Development Support

Although the final solution must be hosted, the repository must also explain how to run the application locally. Provide either Docker Compose, or Kind, Minikube or another local Kubernetes setup.

A typical local startup should be documented through commands such as:

```
cp .env.example .env
docker compose up -d
npm run migrate
npm run seed
npm run dev
```

The exact commands may differ, but the evaluator should be able to reproduce the application from the repository.

11. Testing Requirements

Include automated tests for at least the following scenarios:

1. Incident creation
2. Incident validation failure
3. Dependency traversal
4. Candidate-change ranking
5. MCP tool invocation
6. LangGraph workflow completion
7. Low-confidence analysis path
8. Temporal approval signal
9. Temporal rejection signal
10. Temporal activity retry
11. Duplicate approval handling
12. Remediation failure and escalation
13. Audit-log creation
14. Secret redaction
15. Health and readiness endpoints

Tests may include unit tests, integration tests, workflow tests, API tests and end-to-end tests.

LLM calls should be mocked in automated tests.

Continuous integration

A CI workflow should run the tests whenever code is pushed or a pull request is opened. GitHub Actions is recommended.

12. Documentation Requirements

The repository's README.md must contain the following sections.

README section	Required content
Project Summary	Explain the purpose of the system and the problem being solved.
Architecture	Include a component diagram, request-flow diagram, LangGraph workflow, Temporal workflow and data-flow explanation.
Technology Decisions	Explain the major technology choices, trade-offs and separation of responsibilities.
Setup Instructions	Cover local setup, migrations, sample data, service startup, tests and cloud deployment.
Hosted Environment	Provide the hosted endpoint, health endpoint, API documentation and evaluator usage instructions.
API Examples	Provide working curl, Postman, Bruno or equivalent examples.
Agent Explanation	Describe state, nodes, branches, tool invocation, error handling and iteration limits.
Temporal Explanation	Describe workflows, activities, signals, queries, retries, timeouts and idempotency.
Security	Describe secret management, validation, redaction, authentication and remediation controls.
Assumptions and Limitations	State simplifications, simulated behaviours, unsupported cases, known issues and production-hardening needs.

Technology Decisions Must Explain

- Why each major technology was selected
- How LangGraph and Temporal responsibilities are separated
- Why MCP is used between the agent and operational tools
- How deterministic ranking is separated from LLM reasoning

13. Required Submission

The final submission must include all of the following:

1. GitHub repository URL
2. Hosted application or API URL
3. Health-check URL
4. API documentation
5. Evaluator access instructions
6. Architecture diagram
7. README with setup and deployment instructions
8. Automated test suite
9. CI workflow
10. Sample incident and change data
11. Demonstration instructions

The GitHub repository should show the candidate's development process through meaningful commits.

Commit-history expectation

Avoid submitting the entire solution as a single final commit.

14. Evaluation Walkthrough

The repository should contain a short manual that enables the evaluator to perform the following walkthrough.

1. Verify the Hosted System

```
GET /health

Expected result:
{
  "status": "healthy"
}
```

2. Create the Incident

```
POST /api/incidents
Content-Type: application/json

{
  "incident_id": "INC-1042",
  "service": "order-service",
  "environment": "production",
  "symptom": "Order creation failure rate increased from 1% to 18%",
  "detected_at": "2026-08-04T08:15:00Z"
}
```

3. Start the Analysis

```
POST /api/incidents/INC-1042/analyse
```

4. View Candidate Changes

```
GET /api/incidents/INC-1042/candidates
```

5. View the Analysis

```
GET /api/incidents/INC-1042/analysis
```

6. View Workflow Status

```
GET /api/incidents/INC-1042/workflow

Expected workflow state:
AWAITING_APPROVAL
```

7. Approve the Recommendation

```
POST /api/incidents/INC-1042/approve
Content-Type: application/json

{
  "approved_by": "evaluator@example.com",
  "comment": "Approved for simulated execution."
}
```

8. Verify Workflow Completion

```
GET /api/incidents/INC-1042/workflow
```

The workflow should eventually show either:
RESOLVED

or:
ESCALATED

9. View the Audit Trail

```
GET /api/incidents/INC-1042/audit-log
```

The audit trail should show the complete sequence from incident creation through analysis, approval, remediation and verification.

15. Evaluation Criteria

Area	Weight
End-to-end functional completeness	20%
Code quality and project structure	15%
LangGraph and agent design	15%
Temporal workflow design	15%
MCP implementation	10%
PostgreSQL modelling	10%
Cloud and Kubernetes deployment	10%
Documentation, tests and CI	5%

16. What Will Be Evaluated

Strong submissions will demonstrate:

- A working hosted solution
- A well-organised GitHub repository
- Clear separation between deterministic logic and LLM decisions
- Evidence-based change attribution
- Proper use of MCP tools
- Explicit LangGraph state and branching
- Durable Temporal workflows
- Human approval before remediation
- Idempotent activities and APIs
- Structured auditability
- Secure handling of secrets
- Useful failure handling
- Reproducible deployment
- Clear documentation
- Practical engineering decisions

Evaluation philosophy

A smaller system that works reliably and is clearly documented is preferable to a larger system with incomplete workflows.

17. Bonus Enhancements

Candidates may implement additional capabilities such as:

- A React incident-analysis interface
- Server-Sent Events or WebSocket status updates
- OpenTelemetry tracing
- Multi-level dependency traversal
- A specialised remediation-safety agent
- Policy rules preventing unapproved production actions
- Multiple LLM-provider support
- An incident timeline
- Immutable evidence records
- Authentication and role-based approval
- Prometheus metrics
- Helm-based installation
- GitOps deployment
- Automated hosted-environment deployment through GitHub Actions

Bonus scope

Bonus functionality should not replace or compromise the mandatory workflow.

18. Final Submission Checklist

☐ The solution is hosted and reachable	☐ Temporal is used for durable workflow orchestration
☐ The health endpoint works	☐ Human approval is required before remediation
☐ The complete incident workflow can be demonstrated	☐ Remediation is simulated and auditable
☐ The GitHub repository is accessible to evaluators	☐ Kubernetes or Helm deployment files are included
☐ No passwords or API keys are committed	☐ Automated tests pass
☐ Database migrations are included	☐ GitHub Actions or equivalent CI is configured
☐ Sample data is included	☐ Local setup instructions are complete
☐ MCP tools are implemented and tested	☐ Hosted evaluation instructions are complete
☐ LangGraph is used for agent orchestration	☐ Assumptions and limitations are documented

Ready for evaluation

Submit only after every mandatory item above has been verified against the hosted environment and the GitHub repository.